

Aplicación de Seguimiento de Actividad Física con React Native + Expo + Animaciones.

Proyecto: "MoveMate - Tu compañero/a de movimiento diario".

- **Tecnologías a utilizar:** React Native, Expo SDK 51+, Reanimated 2/3, React Native Gesture Handler, React Native Maps, Expo Location, AsyncStorage, Firebase/Supabase (cloud sync), EAS Build.
- **Objetivo general:** Construir una aplicación móvil de seguimiento de actividad física con animaciones fluidas, mapas interactivos, seguimiento GPS en tiempo real y sincronización en la nube, aplicando buenas prácticas de desarrollo mobile.

Introducción – El mundo del desarrollo móvil moderno.

¿Por qué React Native?

Vivimos pegados al celular. Las apps móviles mueven la economía global: delivery, fitness, fintech, redes sociales. Pero desarrollar para iOS y Android por separado es costoso. React Native resuelve esto: un solo código (JavaScript/TypeScript) que se compila a componentes nativos en ambas plataformas.

Expo es el framework que hace que React Native sea accesible: sin necesidad de configurar Xcode o Android Studio, con recarga en vivo, y con la posibilidad de compilar tu APK en la nube con un solo comando.

¿Por qué MoveMate?

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

El fitness tracking es uno de los casos de uso más apasionantes para mobile:

- **GPS en tiempo real:** Aprenderán a usar sensores del dispositivo.
- **Animaciones fluidas:** Reanimated 2 permite animaciones a 60fps en el hilo nativo.
- **Mapas interactivos:** Integración con Google Maps / MapKit.
- **Sincronización en la nube:** Los datos del usuario se respaldan y sincronizan.
- **Publicación real:** Generarán un APK instalable en cualquier Android.

MoveMate no es solo una app de ejercicios: es una herramienta completa para registrar caminatas, carreras, bicicleteadas, con estadísticas, historial, logros y sincronización.

¿Qué aprenderán?

- **React Native y Expo:** Componentes nativos, estilos, navegación.
- **Reanimated 2:** Animaciones fluidas con worklets y shared values.
- **Gestos:** Pan, tap, pinch, swipe con Gesture Handler.
- **Geolocalización:** Seguimiento GPS con Expo Location.
- **Mapas:** Mapas interactivos con React Native Maps.
- **Cloud Sync:** Firebase o Supabase para persistencia.
- **Publicación:** Generar un APK con EAS Build.

Este proyecto es el cierre perfecto del segundo cuatrimestre. Formará parte de su sistema integrador y deberá ser defendido explicando cada decisión de código.

React Native y Expo Fundamentals.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

React Native traduce componentes JSX a componentes nativos (View → UIView/Android.View, Text → UILabel/TextView). No es un WebView.

Componentes básicos:

```
import { View, Text, StyleSheet, TouchableOpacity, ScrollView, Image } from 'react-native';
```

```
export default function App() {  
  return (  
    <View style={styles.container}>  
      <Text style={styles.title}>¡Hola MoveMate!</Text>  
      <TouchableOpacity onPress={() => alert('Presionado')}>  
        <Text style={styles.button}>Comenzar</Text>  
      </TouchableOpacity>  
    </View>  
  );  
}
```

```
const styles = StyleSheet.create({  
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', backgroundColor:  
    '#0f0f1a' },  
  title: { fontSize: 24, color: 'fff', fontWeight: 'bold' },  
  button: { color: '#4ecdc4', marginTop: 20, fontSize: 18 }  
});
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Diferencias clave con web:

- ``View`` en lugar de ``div``, ``Text`` en lugar de ``p``.
- Estilos con ``StyleSheet.create()`` (objeto JS, no CSS).
- No hay ``className``, se usa ``style``.
- Flexbox es el sistema de layout por defecto.

Expo:

- ``npx create-expo-app MoveMate``
- ``npx expo start`` → abre en el celular con la app "Expo Go".
- Gestión de permisos simplificada (``expo-location``, ``expo-camera``, etc.).
- EAS Build para generar APK/IPA sin necesidad de Mac.

Navegación con Expo Router.

Expo Router usa file-based routing (similar a Next.js).

app/

```
├─ _layout.tsx    // Layout raíz
├─ index.tsx      // Pantalla principal (/)
├─ (tabs)/
|   ├─ _layout.tsx // Tabs
|   └─ home.tsx    // /home
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
| |— history.tsx    // /history
| |— profile.tsx    // /profile
|— activity/
    |— [id].tsx      // /activity/123
```

Ejemplo de tab layout:

```
import { Tabs } from 'expo-router';

import { Icons } from '@expo/vector-icons';

export default function TabsLayout() {

  return (

    <Tabs screenOptions={{ tabBarActiveTintColor: '#4ecdc4' }}>

      <Tabs.Screen name="home" options={{ title: 'Inicio', tabBarIcon: ({ color }) => <Icons
name="home" color={color} size={24} /> }} />

      <Tabs.Screen name="history" options={{ title: 'Historial', tabBarIcon: ({ color }) =>
<Icons name="time" color={color} size={24} /> }} />

      <Tabs.Screen name="profile" options={{ title: 'Perfil', tabBarIcon: ({ color }) => <Icons
name="person" color={color} size={24} /> }} />

    </Tabs>

  );

}
```

Reanimated 2 – Animaciones Fluidas.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

Reanimated 2 ejecuta animaciones en el UI thread, no en el JS thread. Esto significa 60fps garantizados, incluso si el JS está ocupado.

Conceptos clave:

- **Shared Values:** Variables que viven en el UI thread.
- **Worklets:** Funciones que se ejecutan en el UI thread (marcadas con ``worklet``).
- **useAnimatedStyle:** Hook que devuelve un estilo animado.
- **withTiming, withSpring, withDecay:** Funciones de animación.

Ejemplo – Botón que escala al presionar:

```
import Animated, { useSharedValue, useAnimatedStyle, withSpring } from 'react-native-reanimated';
```

```
import { Pressable } from 'react-native';
```

```
export default function AnimatedButton() {
```

```
  const scale = useSharedValue(1);
```

```
  const animatedStyle = useAnimatedStyle(() => ({
```

```
    transform: [{ scale: scale.value }]
```

```
  }));
```

```
  return (
```

```
<Pressable
```

```
onPressIn={() => { scale.value = withSpring(0.9); }}  
onPressOut={() => { scale.value = withSpring(1); }}  
  
>  
  
<Animated.View style={[styles.button, animatedStyle]}>  
  <Text style={styles.buttonText}>Presioname</Text>  
  
</Animated.View>  
  
</Pressable>  
  
);  
}
```

Ejemplo – Animación de entrada:

```
import { useEffect } from 'react';  
  
import Animated, { useSharedValue, useAnimatedStyle, withTiming, withDelay } from 'react-native-reanimated';  
  
const opacity = useSharedValue(0);  
const translateY = useSharedValue(20);  
  
useEffect(() => {  
  opacity.value = withTiming(1, { duration: 500 });  
  translateY.value = withDelay(200, withTiming(0, { duration: 500 }));  
}, []);  
  
const style = useAnimatedStyle(() => ({
```

```
opacity: opacity.value,  
transform: [{ translateY: translateY.value }]  
}});
```

Gestos con React Native Gesture Handler.

Gesture Handler reemplaza los gestos básicos de React Native con una API más potente y compatible con Reanimated.

Ejemplo – Tarjeta deslizable (swipe):

```
import { Gesture, GestureDetector } from 'react-native-gesture-handler';  
  
import Animated, { useSharedValue, useAnimatedStyle, withSpring } from 'react-native-reanimated';  
  
export default function SwipeCard() {  
  const translateX = useSharedValue(0);  
  
  const pan = Gesture.Pan()  
    .onUpdate((e) => { translateX.value = e.translationX; })  
    .onEnd(() => {  
      if (Math.abs(translateX.value) > 100) {  
        // Swipe confirmado  
        translateX.value = withSpring(translateX.value > 0 ? 500 : -500);  
      } else {
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
    translateX.value = withSpring(0);

  }

});

const style = useAnimatedStyle() => ({

  transform: [{ translateX: translateX.value }]

});

return (

  <GestureDetector gesture={pan}>

    <Animated.View style={[styles.card, style]}>

      <Text>Desliza esta tarjeta</Text>

    </Animated.View>

  </GestureDetector>

);

}
```

Geolocalización con Expo Location.

```
import * as Location from 'expo-location';

// Pedir permisos

const { status } = await Location.requestForegroundPermissionsAsync();

if (status !== 'granted') {
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
    alert('Necesitamos permiso de ubicación');

    return;
}

// Obtener posición actual

const location = await Location.getCurrentPositionAsync({
  accuracy: Location.Accuracy.High
});

console.log(location.coords.latitude, location.coords.longitude);

// Suscribirse a cambios (seguimiento en tiempo real)

const subscription = await Location.watchPositionAsync(
  {
    accuracy: Location.Accuracy.High,
    TimeInterval: 1000,    // cada 1 segundo
    distanceInterval: 5    // o cada 5 metros
  },
  (newLocation) => {
    console.log(newLocation.coords);
  }
);

// No olvidar: subscription.remove() al desmontar
```

Mapas con React Native Maps.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
import MapView, { Polyline, Marker } from 'react-native-maps';
```

```
<MapView  
  style={{ flex: 1 }}  
  initialRegion={{  
    latitude: -34.6037,  
    longitude: -58.3816,  
    latitudeDelta: 0.01,  
    longitudeDelta: 0.01  
  }}  
  showsUserLocation  
>  
  
  <Polyline  
    coordinates={route} // Array de {latitude, longitude}  
    strokeColor="#4ecdc4"  
    strokeWidth={4}  
  />  
  
  <Marker coordinate={startPoint} title="Inicio" />  
</MapView>
```

Sincronización en la Nube.

Opciones recomendadas:

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Firebase (Firestore):** simple, con autenticación incluida.
- **Supabase:** alternativa open source basada en Postgres.
- **API propia:** en Node.js/Express desplegada en Vercel/Render.

Ejemplo con Firebase:

```
import { initializeApp } from 'firebase/app';

import { getFirestore, collection, addDoc } from 'firebase/firestore';

const app = initializeApp({ /* config */ });

const db = getFirestore(app);

async function saveActivity(activity) {

  await addDoc(collection(db, 'activities'), {

    ...activity,

    userId: currentUser.uid,

    createdAt: new Date()

  });

}
```

Publicación con EAS Build.

EAS (Expo Application Services) permite compilar tu app en la nube:

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
npm install -g eas-cli
```

```
eas login
```

```
eas build:configure
```

```
eas build -p android --profile preview
```

Esto genera un APK descargable que se puede instalar en cualquier Android.

Ejemplo: "App de Recetas con Animaciones y Cámara".

Para que vean cómo se estructura una app React Native completa, construyamos un mini ejemplo: una app de recetas que permite agregar recetas con foto y tiene animaciones.

Estructura:

```
app/
```

```
├─ _layout.tsx
```

```
├─ index.tsx          // Lista de recetas
```

```
├─ add-recipe.tsx     // Formulario con cámara
```

```
└─ recipe/[id].tsx    // Detalle
```

```
components/
```

```
├─ RecipeCard.tsx     // Tarjeta animada
```

```
└─ FloatingButton.tsx // Botón con animación
```

`app/index.tsx` – Lista con animaciones:

```
import { View, Text, FlatList, StyleSheet } from 'react-native';

import { useState } from 'react';

import Animated, { FadeInRight } from 'react-native-reanimated';

import { useRouter } from 'expo-router';

import RecipeCard from '../components/RecipeCard';

import FloatingButton from '../components/FloatingButton';

export default function Home() {

  const [recipes, setRecipes] = useState([

    { id: '1', title: 'Pizza Margarita', image: '...' },

    { id: '2', title: 'Pasta Carbonara', image: '...' }

  ]);

  const router = useRouter();

  return (

    <View style={styles.container}>

      <FlatList

        data={recipes}

        keyExtractor={(item) => item.id}

        renderItem={({ item, index }) => (

          <Animated.View entering={FadeInRight.delay(index * 100).duration(400)}>

            <RecipeCard recipe={item} onPress={() => router.push(`/recipe/${item.id}`} />

          </Animated.View>

        )}

      />

    </View>

  )}
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
    />

    <FloatingButton onPress={() => router.push('/add-recipe')} />

  </View>

);

}
```

```
const styles = StyleSheet.create({

  container: { flex: 1, backgroundColor: '#0f0f1a', padding: 16 }

});
```

`components/FloatingButton.tsx` – Botón animado:

```
import { Pressable, StyleSheet } from 'react-native';

import Animated, { useSharedValue, useAnimatedStyle, withSpring } from 'react-native-reanimated';

import { Ionicons } from '@expo/vector-icons';

export default function FloatingButton({ onPress }: { onPress: () => void }) {

  const scale = useSharedValue(1);

  const rotation = useSharedValue(0);

  const style = useAnimatedStyle(() => ({

    transform: [{ scale: scale.value }, { rotate: `${rotation.value}deg` }]

  }));

  return (
```

```
<Pressable

  onPressIn={() => { scale.value = withSpring(0.85); rotation.value = withSpring(45); }}

  onPressOut={() => { scale.value = withSpring(1); rotation.value = withSpring(0); }}

  onPress={onPress}

>

  <Animated.View style={[styles.fab, style]}>

    <Icons name="add" size={32} color="#fff" />

  </Animated.View>

</Pressable>

);

}
```

```
const styles = StyleSheet.create({

  fab: {

    position: 'absolute',

    bottom: 24,

    right: 24,

    width: 60,

    height: 60,

    borderRadius: 30,

    backgroundColor: '#4ecdc4',

    justifyContent: 'center',

    alignItems: 'center',

    shadowColor: '#4ecdc4',
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
shadowOpacity: 0.5,  
shadowRadius: 10,  
shadowOffset: { width: 0, height: 4 },  
elevation: 8  
}  
});
```

Este ejemplo demuestra la estructura típica de una app Expo Router, con animaciones de entrada, botón flotante animado y navegación. En MoveMate aplicarán patrones similares pero con GPS, mapas y Reanimated para el tracking en vivo.

Proyecto MoveMate – Descripción y Requisitos.

Descripción General.

MoveMate es una aplicación móvil de seguimiento de actividad física que permite:

- Registrar caminatas, carreras y bicicleteadas en tiempo real.
- Visualizar el recorrido en un mapa interactivo con GPS.
- Ver estadísticas y logros con animaciones fluidas.
- Sincronizar los datos en la nube.
- Generar un APK instalable para Android.

Requisitos Funcionales (ID).

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

ID	Requerimiento	Descripción
M1	Pantalla de inicio	Dashboard con resumen del día: pasos, distancia, calorías. Animaciones de entrada.
M2	Iniciar actividad	Botón flotante animado que inicia una nueva sesión. Selector de tipo (caminata, carrera, bici).
M3	Seguimiento GPS en vivo	Obtener posición en tiempo real con Expo Location, dibujar la ruta sobre el mapa.
M4	Mapa interactivo	Mostrar el recorrido con `Polyline`, marcador del usuario, controles de zoom.
M5	Animaciones fluidas	Usar Reanimated 2 para: botón pulsante, transiciones entre pantallas, animaciones de estadísticas (números que suben).
M6	Historial	Lista de actividades anteriores con filtros por tipo y fecha. Cada item es una tarjeta deslizable con gestos.
M7	Sincronización cloud	Guardar actividades en Firebase/Supabase y recuperarlas al iniciar sesión.
M8	Generar APK	Compilar la app con EAS Build y publicar el APK descargable.
M9	Gestos	Al menos un gesto complejo (swipe para eliminar, pinch para zoom en mapa, long-press para opciones).
M10	Modo oscuro/claro	Soporte de temas con detección automática.

Objetivos del Proyecto.

- **Técnico:** Dominar React Native, Expo y Reanimated.
- **Arquitectónico:** Estructurar una app mobile con navegación, estado y persistencia.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Práctico:** Usar sensores del dispositivo (GPS) y APIs nativas.
- **Profesional:** Publicar un APK real y sincronizar con backend.

Comandos clave para empezar:

```
npx create-expo-app MoveMate -t expo-template-blank-typescript
```

```
cd MoveMate
```

```
npx expo install react-native-reanimated react-native-gesture-handler
```

```
npx expo install expo-location react-native-maps
```

```
npx expo install expo-router react-native-safe-area-context react-native-screens
```

Criterios de Evaluación.

Evaluación Continua (60% – Durante la clase).

- **Conexión y participación:** Estar en Live Share, interactuar, hacer preguntas.
- **Avance parcial:** Antes de finalizar la clase, subir un `.zip` del proyecto (excluyendo `node\_modules`) o enlace a repositorio GitHub/Drive con el avance. Debe incluir:
 - Proyecto Expo con dependencias instaladas.
 - Pantalla de inicio funcional con al menos una animación de Reanimated.
 - Permisos de ubicación solicitados correctamente.
 - Al menos un componente con gesto funcionando.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- **Calidad de código:** Uso de TypeScript, componentes modulares, nombres claros, buenas prácticas de React Native.

Evaluación Final (40% – Entrega hasta el martes siguiente).

- **Funcionalidad:** Cumplir con al menos 6 de los 10 requisitos (M1 a M10).
- **Animaciones:** Al menos 3 animaciones fluidas con Reanimated 2.
- **GPS:** El seguimiento GPS debe funcionar y mostrar la ruta en el mapa.
- **Cloud:** Los datos deben persistir en Firebase/Supabase o API propia.
- **APK:** Debe entregarse un APK generado con EAS Build, instalable en Android.
- **Despliegue del backend:** El backend (API o Firebase), debe estar desplegado en Vercel/Netlify/Render.
- **Informe técnico:** Documento con arquitectura, decisiones, problemas y soluciones.
- **Código limpio:** TypeScript, buenas prácticas, comentarios en partes complejas.

Entregables.

Durante la clase (Avance Parcial).

1. Enlace de Live Share: Compartir en el comentario privado de la plataforma antes de empezar a codificar.
2. Avance Parcial: Subir a la plataforma:
 - `.zip` del proyecto (sin `node\_modules`), O
 - Enlace a repositorio GitHub con el avance, O
 - Enlace a Google Drive con el archivo comprimido.

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

3. Capturas de pantalla del avance funcionando (pantalla de inicio, animación, permisos).

Entrega Final (Martes siguiente).

1. Repositorio GitHub: Repositorio público con el código completo y `README.md` documentado.

2. APK: Archivo `.apk` descargable (puede subirse a Drive o a GitHub Releases). Subir ambas versiones: avance parcial y final.

3. Backend desplegado: URL del backend en Vercel/Netlify/Render/Firebase.

4. Informe Técnico: En PDF o Markdown, que incluya:

- Introducción y objetivos.
- Explicación de la arquitectura (pantallas, componentes, stores).
- Capturas de pantalla de cada funcionalidad.
- Capturas de las animaciones (GIF o video corto).
- Dificultades encontradas y soluciones.
- Conclusiones y aprendizajes.
- Enlace al APK y al backend desplegado.

5. Video demo: Grabación de 2-3 minutos mostrando la app en un dispositivo real.

Sistema Integrador y Defensa Final.

Recordatorio clave: MoveMate no es un proyecto aislado. Forma parte del sistema integrador del segundo cuatrimestre, donde deberán combinar todas las tecnologías vistas durante el año (HTML/CSS/JS, frameworks web, frameworks mobile, bases de datos, APIs, cloud) en un proyecto unificado, o bien acoplar todo lo aprendido a su proyecto final.

Defensa final: Al final del cuatrimestre, defenderán su proyecto final explicando:

- La arquitectura y decisiones técnicas.
- Cómo se integran las diferentes tecnologías (web + mobile + backend).
- El código (cada línea importante).
- Los desafíos y cómo los resolvieron.